Universidade Federal Fluminense
LACOP, Laboratório de Comunicações Ópticas

Decifrando o código: temperatura e conforto de ambiente

Explicação linha por linha para quem está aprendendo Python, com a leitura dos resultados obtidos

Antes de começar: o que este código faz

Este documento percorre, com calma e em detalhe, um código escrito em Python que usa dados de sensores de ambiente para resolver dois problemas ao mesmo tempo. O texto foi pensado para quem está dando os primeiros passos em programação, então cada linha é explicada no seu próprio contexto, sem pressupor conhecimento prévio de bibliotecas de inteligência artificial.

A base de dados vem de um sensor instalado em salas do laboratório, que registrou, a cada quinze minutos, a temperatura, a umidade, a luminosidade, se o ar condicionado estava ligado, uma estimativa de quantas pessoas havia no ambiente, e um rótulo de conforto térmico. São 576 leituras ao todo.

O código resolve dois problemas distintos a partir desses mesmos dados. O primeiro é uma regressão: prever a temperatura em graus, que é um número contínuo. O segundo é uma classificação: prever o conforto do ambiente, que é uma categoria, frio, confortável ou quente. Entender que são duas tarefas diferentes, com modelos e métricas diferentes, é a chave para ler o código todo.

Dois modelos são usados. Para a temperatura, uma **Regressão Linear**, que traça a melhor relação matemática entre as entradas e o valor a prever. Para o conforto, uma **Árvore de Decisão**, que aprende uma sequência de perguntas para separar as categorias. Veremos cada um no seu lugar.

Bloco 1: as importações

Todo programa em Python começa trazendo as ferramentas que vai usar. Pense nas importações como abrir a caixa de ferramentas e pegar só o que será necessário. Cada linha abaixo traz uma peça específica.

```
import pandas as pd
```

Traz a biblioteca `pandas` e dá a ela o apelido curto `pd`. O pandas é a ferramenta que abre o arquivo de dados e o organiza como uma tabela, com linhas e colunas. O apelido serve só para você escrever `pd` em vez de `pandas` o tempo todo.

```
from sklearn.model_selection import train_test_split
```

Da biblioteca `sklearn`, o scikit-learn, traz a função `train_test_split`. Ela separa os dados em duas partes, uma para o modelo treinar e outra, guardada, para testar se ele aprendeu de verdade.

```
from sklearn.linear_model import LinearRegression
```

Traz o modelo `LinearRegression`, a Regressão Linear, que será usado para prever a temperatura, um número.

```
from sklearn.tree import DecisionTreeClassifier
```

Traz o modelo `DecisionTreeClassifier`, a Árvore de Decisão, que será usado para prever o conforto, uma categoria.

```
from sklearn.metrics import mean_absolute_error,
     r2_score, accuracy_score, classification_report
```

Traz quatro ferramentas de avaliação, que medem o desempenho dos modelos. `mean_absolute_error` e `r2_score` servem para a regressão da temperatura. `accuracy_score` e `classification_report` servem para a classificação do conforto. Cada uma será explicada quando aparecer.

```
from sklearn.preprocessing import StandardScaler,
     OneHotEncoder, LabelEncoder
```

Traz três ferramentas que preparam os dados antes de o modelo aprender. `StandardScaler` ajusta a escala dos números, `OneHotEncoder` converte categorias em números, e `LabelEncoder` transforma o alvo de texto em número. Tudo isso é detalhado adiante.

```
from sklearn.compose import ColumnTransformer
```

Traz o `ColumnTransformer`, que aplica tratamentos diferentes a grupos diferentes de colunas de uma só vez.

```
from sklearn.pipeline import Pipeline
```

Traz o `Pipeline`, que encadeia a preparação dos dados e o modelo em uma sequência única e organizada.

Bloco 2: carregar o arquivo e olhar os dados

```
df = pd.read_csv('/content/ambiente_temperatura_umidade (1).csv')
```

Esta linha lê o arquivo de dados, que está no formato CSV, uma tabela em texto, e guarda tudo numa variável chamada `df`. O nome `df` é uma abreviação de DataFrame, que é como o pandas chama uma tabela. O caminho `/content/` é a pasta padrão do Google Colab, onde ficam os arquivos enviados. Repare no `(1)` no nome do arquivo: ele precisa bater exatamente com o nome real, senão o programa não encontra o arquivo.

```
print("Dados carregados com sucesso. Primeiras 5 linhas:")
display(df.head())
```

A primeira linha apenas escreve uma mensagem na tela. A segunda, `df.head()`, mostra as cinco primeiras linhas da tabela, e `display` as exibe de forma organizada no Colab. É uma espiada rápida para confirmar que os dados foram lidos corretamente e têm a aparência esperada.

```
print("\nInformações gerais do DataFrame:")
df.info()
```

O `df.info()` imprime um resumo técnico da tabela: quantas linhas existem, quais são as colunas, quantos valores preenchidos cada uma tem, e o tipo de cada coluna. O `\n` na mensagem apenas pula uma linha antes do texto, para deixar a saída mais limpa. Essa conferência inicial ajuda a perceber problemas, como uma coluna que deveria ser número mas veio como texto.

Bloco 3: preparar os dados

Antes de qualquer modelo aprender, os dados precisam estar no formato certo. Este bloco é onde isso acontece, e cada decisão aqui afeta o resultado final. Vamos com calma.

Converter a data e hora

```
df['timestamp'] = pd.to_datetime(df['timestamp'])
```

A coluna `timestamp` guarda a data e a hora de cada leitura, mas veio como texto puro. Esta linha a converte para um tipo especial de data e hora que o Python entende de verdade. Sem essa conversão, não seria possível extrair a hora ou o dia da semana, que é o próximo passo.

Criar novas colunas a partir da data

As três linhas a seguir criam colunas novas, derivadas do timestamp. Isso se chama engenharia de atributos, e é uma das partes mais inteligentes do código: em vez de entregar a data crua ao modelo, o código extrai dela pistas úteis que ajudam a prever a temperatura e o conforto.

```
df['hora'] = df['timestamp'].dt.hour
```

Cria a coluna `hora`, pegando apenas a hora do dia (de 0 a 23) de cada leitura. Faz sentido, porque a temperatura muda ao longo do dia, sobe à tarde e cai de madrugada.

```
df['dia_da_semana'] = df['timestamp'].dt.dayofweek
```

Cria a coluna `dia_da_semana`, onde segunda é 0 e domingo é 6. Pode ajudar porque o uso de uma sala muda entre dias úteis e fins de semana, afetando a temperatura e o conforto.

```
df['dia_do_ano'] = df['timestamp'].dt.dayofyear
```

Cria a coluna `dia_do_ano`, um número de 1 a 365. Captura a época do ano, útil para refletir variações sazonais.

Separar as entradas e os dois alvos

```
features = ['hora', 'dia_da_semana', 'dia_do_ano', 'sala',
    'umidade_pct', 'luminosidade_lux', 'ar_condicionado',
    'pessoas_estimadas']
```

Cria uma lista chamada `features` com os nomes das colunas que servirão de entrada, as pistas que os modelos vão usar para prever. Repare que inclui as três colunas de tempo recém-criadas, mais a sala, a umidade, a luz, o ar condicionado e a estimativa de pessoas. A coluna `timestamp` original ficou de fora, porque já foi transformada em hora, dia da semana e dia do ano.

```
X = df[features]
```

Cria `X`, que contém apenas as colunas de entrada listadas acima. Por convenção, o `X` maiúsculo representa as entradas do modelo.

```
y_temp = df['temperatura_c']
```

Cria `y_temp`, o primeiro alvo: a temperatura em graus. Este é o que o modelo de regressão vai tentar prever.

```
y_conforto = df['conforto']
```

Cria `y_conforto`, o segundo alvo: o conforto do ambiente. Este é o que o modelo de classificação vai tentar prever. Repare que o código guarda dois alvos separados, porque vai treinar dois modelos diferentes a partir das mesmas entradas.

Definir como cada tipo de coluna será tratado

```
categorical_features = ['sala']
```

Lista as colunas categóricas, ou seja, as que guardam nomes em vez de números. Aqui só a coluna `sala`, que contém textos como lab_optica.

```
numerical_features = ['hora', 'dia_da_semana',
    'dia_do_ano', 'umidade_pct', 'luminosidade_lux',
    'ar_condicionado', 'pessoas_estimadas']
```

Lista as colunas numéricas, as que já são números. Essa separação é necessária porque números e categorias precisam de preparos diferentes, feitos no passo seguinte.

Montar o preparador de dados

```
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'),
         categorical_features)
    ])
```

Este é o coração da preparação. O `ColumnTransformer` é um organizador que aplica dois tratamentos ao mesmo tempo, cada um ao seu grupo de colunas.

Ao grupo numérico, aplica o `StandardScaler`. Ele coloca todas as colunas de números numa escala comparável. Isso importa porque as colunas têm faixas muito diferentes: a luminosidade vai a centenas, enquanto a hora vai só até 23. Sem esse ajuste, o modelo de regressão daria peso exagerado às colunas de números grandes, apenas por serem grandes.

Ao grupo categórico, aplica o `OneHotEncoder`. Como o modelo só entende números, ele transforma cada valor da coluna sala em uma coluna própria de zeros e uns. O parâmetro `handle_unknown='ignore'` evita que o programa quebre caso apareça, no teste, uma sala que não estava no treino.

Bloco 4: o modelo de temperatura (regressão)

A partir daqui, o código trata do primeiro problema: prever a temperatura, um número. Isso é uma regressão.

Dividir em treino e teste

```
X_train_temp, X_test_temp, y_train_temp, y_test_temp =
    train_test_split(X, y_temp, test_size=0.2,
    random_state=42)
```

Separa os dados em quatro partes: entradas de treino e de teste, e alvos de treino e de teste. O `test_size=0.2` reserva vinte por cento das leituras para teste, dados que o modelo não vê durante o aprendizado e que servem para avaliá-lo com honestidade. O `random_state=42` faz a divisão ser sempre a mesma a cada execução, o que torna o resultado reprodutível.

Montar e treinar o modelo

```
model_temp = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])
```

Cria um `Pipeline`, que encadeia o preparo dos dados e o modelo em uma sequência. Quando o código mandar treinar, os dados passam primeiro pelo `preprocessor` (a normalização e a conversão de categorias) e só então chegam à `LinearRegression`. A vantagem é a consistência: o mesmo tratamento aplicado ao treino é aplicado automaticamente ao teste, sem risco de esquecer um passo.

```
model_temp.fit(X_train_temp, y_train_temp)
```

Esta é a linha onde o aprendizado acontece. O `fit` mostra ao modelo as entradas de treino e as temperaturas corretas, e a Regressão Linear ajusta seus parâmetros para melhor relacionar uma coisa à outra. Depois desta linha, o modelo está treinado.

Avaliar o modelo de temperatura

```
y_pred_temp = model_temp.predict(X_test_temp)
```

Usa o modelo treinado para prever a temperatura das leituras de teste, guardando os palpites em `y_pred_temp`. São esses palpites que serão comparados com as temperaturas reais.

```
print(f"Erro Médio Absoluto (MAE): ...")
print(f"R-squared (R2): ...")
```

Estas linhas calculam e imprimem duas notas. O `MAE`, erro médio absoluto, diz de quantos graus o modelo erra em média. O `R2`, R ao quadrado, diz qual fração da variação da temperatura o modelo consegue explicar, numa escala de 0 a 1, onde mais perto de 1 é melhor. Os valores obtidos são interpretados mais adiante.

Bloco 5: o modelo de conforto (classificação)

Agora o código trata do segundo problema: prever o conforto, uma categoria. Isso é uma classificação, e exige um passo a mais, porque o alvo veio como texto.

Converter o alvo de texto em número

```
le = LabelEncoder()
y_conforto_encoded = le.fit_transform(y_conforto)
```

A coluna conforto vem escrita como texto: frio, confortavel, quente. O modelo, porém, só entende números. O `LabelEncoder`, guardado na variável `le`, faz essa ponte, atribuindo um número a cada categoria. O `fit_transform` aprende quais categorias existem e, ao mesmo tempo, troca cada nome por um número, guardando o resultado em `y_conforto_encoded`. Guardar o `le` é importante, pois ele permitirá, no fim, traduzir os números de volta para os nomes na hora de exibir o relatório.

Dividir em treino e teste, com cuidado extra

```
X_train_conforto, X_test_conforto, y_train_conforto,
    y_test_conforto = train_test_split(X,
    y_conforto_encoded, test_size=0.2, random_state=42,
    stratify=y_conforto_encoded)
```

Divide os dados como antes, mas com um detalhe decisivo: o `stratify=y_conforto_encoded`. Ele garante que a proporção de cada categoria de conforto seja a mesma no treino e no teste. Sem isso, o conjunto de teste poderia, por azar, ficar com poucas leituras de uma categoria rara, como frio, e a avaliação daquela categoria ficaria distorcida. Em dados com classes de tamanhos diferentes, esse cuidado protege a confiabilidade das métricas.

Montar e treinar o modelo de conforto

```
model_conforto = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', DecisionTreeClassifier(random_state=42))
])
model_conforto.fit(X_train_conforto, y_train_conforto)
```

Cria outro pipeline, agora com uma `DecisionTreeClassifier`, a Árvore de Decisão, no lugar da regressão. A árvore aprende uma sequência de perguntas do tipo maior ou menor que um valor para separar as categorias de conforto. O `fit` treina esse modelo com os dados de conforto. Repare que o mesmo `preprocessor` é reaproveitado, o que mantém a preparação idêntica à do outro modelo.

Avaliar o modelo de conforto

```
y_pred_conforto = model_conforto.predict(X_test_conforto)
```

Prevê o conforto das leituras de teste, guardando os palpites em `y_pred_conforto`.

```
print(f"Acurácia: ...")
print("Relatório de Classificação:\n",
      classification_report(..., target_names=le.classes_))
```

Imprime a acurácia, a proporção geral de acertos, e o relatório de classificação, que detalha o desempenho em cada categoria. O `target_names=le.classes_` faz o relatório mostrar os nomes reais, frio, confortavel e quente, em vez de números, usando aquele conversor que foi guardado antes.

Bloco 6: lendo os resultados obtidos

Com o código executado, a tela mostrou números concretos. Esta seção explica o que cada um significa e o conecta com as decisões tomadas no código.

Primeiro, o que os dados revelaram

O `df.info()` confirmou pontos importantes. A tabela tem 576 leituras e 8 colunas originais, e nenhuma coluna tem valores faltando, todas aparecem como 576 non-null. Isso é um bom sinal, pois significa que não há buracos nos dados que precisassem ser preenchidos. Também mostrou que `timestamp`, `sala` e `conforto` são do tipo object, ou seja, texto, enquanto as medições são números. Foi justamente por isso que o código precisou converter o timestamp e codificar a sala e o conforto.

O modelo de temperatura: um resultado muito bom

A avaliação da regressão trouxe dois números:

Métrica	Valor obtido
Erro Médio Absoluto (MAE)	0,78 graus
R ao quadrado (R2)	0,92

O **MAE de 0,78** quer dizer que, em média, a temperatura prevista pelo modelo erra por menos de um grau em relação à real. Para prever temperatura de ambiente, isso é um erro pequeno e aceitável. Na prática, se o modelo diz que a sala está com 22 graus, o valor verdadeiro costuma estar entre 21 e 23.

O **R2 de 0,92** é ainda mais revelador. Ele significa que o modelo consegue explicar noventa e dois por cento da variação da temperatura a partir das entradas que recebeu, a hora, a umidade, a luz, o ar condicionado e as demais. Como a escala vai até 1, um valor de 0,92 indica um ajuste muito forte. Em outras palavras, as pistas escolhidas no código realmente carregam quase toda a informação necessária para prever a temperatura. As colunas de tempo criadas no bloco 3 provavelmente tiveram papel importante nesse desempenho.

O modelo de conforto: bom, e honesto

A avaliação da classificação trouxe a acurácia e um relatório por categoria. Primeiro, a acurácia:

A **acurácia foi de 0,83**, ou seja, o modelo acertou o conforto em oitenta e três por cento das leituras de teste. É um bom número, mas, como vimos em outros projetos, a acurácia sozinha pode enganar quando as categorias têm tamanhos diferentes. Por isso o relatório por categoria é tão valioso. Ele está reproduzido abaixo.

Categoria	Precisão	Recall	F1	Amostras
confortavel	0,80	0,83	0,82	54
frio	0,88	0,79	0,83	19
quente	0,84	0,84	0,84	43
Acurácia geral			0,83	116

Vale entender cada coluna desse relatório, pois cada uma conta uma parte da história.

- **Precisão.** Entre as leituras que o modelo chamou de uma categoria, quantas eram daquela categoria de verdade. Para frio, a precisão foi de 0,88, a mais alta, o que indica que, quando o modelo diz frio, ele costuma estar certo.
- **Recall.** Entre todas as leituras que eram de fato de uma categoria, quantas o modelo conseguiu encontrar. Para frio, o recall foi de 0,79, o mais baixo, o que significa que o modelo deixou passar algumas leituras frias, classificando-as como outra coisa.
- **F1.** Um equilíbrio entre precisão e recall, resumido em um número. As três categorias ficaram entre 0,82 e 0,84, bem parecidas, o que mostra um modelo equilibrado, sem favorecer demais nenhuma categoria.
- **Amostras (support).** Quantas leituras de cada categoria havia no teste. Aqui aparece um ponto importante: confortavel teve 54, quente teve 43, mas frio teve apenas 19. A categoria frio é a mais rara.

E aqui o código mostra seu acerto. Lembra do `stratify` no bloco 5? Foi graças a ele que a categoria frio, mesmo sendo rara, apareceu de forma proporcional no teste, com 19 amostras, permitindo avaliá-la de maneira justa. Sem esse cuidado, o frio poderia quase sumir do teste, e o relatório não teria sentido para essa categoria.

Conclusão: os dois modelos funcionaram

Juntando tudo, o código cumpriu bem os dois objetivos. O modelo de temperatura prevê com erro menor que um grau e explica a grande maioria da variação, um resultado forte. O modelo de conforto acerta mais de oitenta por cento das vezes e, mais importante, se mantém equilibrado entre as três categorias, inclusive na mais rara. As escolhas de preparação dos dados, a normalização dos números, a conversão das categorias, a criação das colunas de tempo e o uso do stratify, foram o que sustentou esses resultados.

Uma observação honesta, que o próprio código traz nos comentários finais: para prever a temperatura daqui a uma hora, e não a temperatura atual, seria preciso um modelo mais sofisticado, voltado a séries temporais, que levasse em conta a dependência entre instantes. O código atual prevê a temperatura do próprio instante a partir das outras medições, o que é diferente de prever o futuro, mas é um excelente primeiro passo para entender como os dados se relacionam.
